

Salad Box Ordering App – Developer Handoff Spec

Client: Salad Box (Kyanja, Kampala, Uganda)

Purpose: Migrate an existing, working React ordering app + admin dashboard from a Claude Artifact prototype into a permanently hosted, production-ready web app on Netlify (or Vercel).

Version note: This spec reflects the current build as of June 2026, including delivery date selection, payment method, and order cancellation features added after the initial version.

1. What Already Exists

A fully functional React app (single file, `salad-box-orders.jsx`) has been built and tested as a working prototype. It includes:

Customer order form

- Salad selection with quantity steppers, covering the real Salad Box menu:
 - Chicken Salad – 20,000 UGX
 - Beef Salad – 20,000 UGX
 - Egg & Meat Salad – 25,000 UGX
 - Pasta Salad – 25,000 UGX
 - 50:50 Meat – 20,000 UGX
- Customer name and phone number fields
- Delivery zone selection (5 zones, Kyanja-based, fees from 3,000–10,000 UGX depending on distance – see Section 7)
- Delivery address / landmark field and optional notes field
- **Delivery date picker** – customer selects "Tomorrow" or any of the next 7 days
- **Payment method selection** – Cash on Delivery, or Mobile Money (with the business's MoMo number shown inline: 0765119166)
- Live price calculation (items + delivery fee = grand total)
- Order confirmation screen showing the full order summary, selected delivery date, and payment method

- **Customer-side order cancellation** — right after confirming, the customer can cancel their own order (with a confirmation prompt) if placed in error

Internal admin dashboard

- **PIN-gated access** — tapping "Dashboard" prompts for a 4-digit PIN before showing any order data; stays unlocked for the session until the page is reloaded. Currently a simple frontend-only check (PIN is a hardcoded constant in the code), which is a basic deterrent but not real security — see Section 2 for the recommended upgrade.
- Orders grouped and filterable **by delivery date** (not order placement date) — shown as date pills across the top, e.g. "Tomorrow," "Wed 24 Jun"
- Order count and revenue summary for the currently selected delivery date (revenue excludes cancelled orders)
- Status filters: All / Pending / Delivered / **Cancelled**
- Each order card shows: customer name, phone, items ordered, delivery zone +

address, payment method, order timestamp, and total

- **Mark as delivered** and **Cancel order** actions per order (cancel requires confirmation)
- Cancelled orders can be **restored** back to pending if cancelled by mistake
- **Print/Export Day button** — generates a clean, printable order sheet for the currently selected delivery date (opens in a new window/tab, triggers the browser print dialog, can be saved as PDF). Includes a numbered table of all orders for that day with customer, items, delivery zone, payment method, total, and delivery status; cancelled orders are shown struck-through and excluded from the revenue total.

Branding

- Uses the real Salad Box logo (vegetable/leaf mark), embedded directly in the code as a base64 data URI — no external image hosting dependency
- Color palette: green `#4e9d35`, black `#1a1a1a`, white/light-grey neutrals

- Fonts: Anton (bold headers, closest practical match to the brand's "Berthold Block"), Sniglet (body text), Dancing Script (script accents, matching the brand's "Moontime" style), loaded via Google Fonts `@import`

The only thing that needs to change for production is the data storage layer. All UI, business logic, pricing, zones, and branding can be carried over as-is.

The prototype file currently uses a mock `window.storage` API (`get / set / list`) that **only exists in the Claude Artifacts sandbox** — this must be replaced with a real backend per the spec below.

2. Scope of Work

1. Set up a GitHub repository containing the React app.
2. Replace all `window.storage` calls with a real database (Supabase recommended — see below).
3. Deploy the app to Netlify (or Vercel) with continuous deployment from GitHub.

4. Connect a custom domain (client will provide domain, e.g. `saladboxkampala.com`) – optional; can launch on the free `*.netlify.app` subdomain first.
5. Basic end-to-end testing: confirm orders save correctly, dashboard reflects new orders under the correct delivery date, status updates (delivered/cancelled/restored) persist, print sheet generates correctly.
6. Handover: provide the client with login credentials for Netlify and the Supabase dashboard, plus a short "how to make small text/price changes yourself" guide if feasible.

Out of scope (flag to client if requested later):

- Online payment integration / automatic Mobile Money confirmation – currently the app just displays the MoMo number and asks the customer to keep their transaction message; no API-based payment verification exists yet.
- SMS/WhatsApp notifications on new orders – would likely use Supabase + a webhook to Twilio or Africa's Talking.

- Subscription/monthly-plan ordering — discussed with the client as a future phase, not part of this build.

Strongly recommended upgrade (not strictly required, but should be raised with the client):

- The dashboard currently uses a **hardcoded PIN check in the frontend code** (`DASHBOARD_PIN` constant) — functional as a basic deterrent, but technically visible to anyone who inspects the page source, and provides no real session security. Before this handles real customer data at scale, recommend replacing it with proper authentication — e.g. Supabase Auth with a single staff account, or a server-side environment-variable-based PIN check via a Netlify Function, so the PIN is never shipped to the browser. This is a small, well-scoped addition (~1–2 hours) on top of the main migration.

3. Recommended Stack

Layer	Recommendation	Why
Frontend hosting	Netlify (free tier)	Auto-deploys from GitHub, generous free tier, simple setup
Database	Supabase (free tier)	Free Postgres database with a simple JS client, generous free tier, easy to query/filter orders by date for the dashboard
Source control	GitHub (free)	Required for Netlify's continuous deployment

Layer	Recommendation	Why
Domain (optional)	Client-purchased domain via any registrar	Connect via Netlify's domain settings

4. Database Schema (Supabase / Postgres)

Create a single `orders` table:

```
create table orders (  
  id text primary key,  
  created_at timestamp with time zone  
default now(),  
  delivery_date date not null,  
  -- the day the customer wants delivery  
  delivery_date_label text,  
  -- human-readable label, e.g. "Tomorrow  
  -- Sun 21 Jun"  
  customer_name text not null,  
  phone text not null,  
  zone_id text not null,  
  zone_name text not null,  
  delivery_fee integer not null,  
  address text not null,
```

```

    notes text,
    payment_method text not null,
    -- 'cash' | 'momo'
    payment_method_label text,
    items jsonb not null,
    -- array of {id, name, price, qty}
    items_total integer not null,
    grand_total integer not null,
    status text not null default
    'pending'      -- 'pending' |
    'delivered' | 'cancelled'
);

-- Index for the dashboard's date-based
filtering
create index idx_orders_delivery_date
on orders(delivery_date);

```

No authentication table is needed unless dashboard login is added (strongly recommended – see Out of Scope).

5. Code Changes Required

In `salad-box-orders.jsx`, all calls to `window.storage` need to be replaced with Supabase client calls. There are three call sites: saving a new order, loading all orders for the

dashboard, and updating an order's status (used for delivered/cancelled/restored).

Setup

```
import { createClient } from
  '@supabase/supabase-js';
const supabase =
  createClient(SUPABASE_URL,
    SUPABASE_ANON_KEY);
```

Store `SUPABASE_URL` and `SUPABASE_ANON_KEY` in Netlify's environment variables — never hardcode them.

Saving a new order — replace:

```
await window.storage.set(order.id,
  JSON.stringify(order), true);
```

With:

```
await supabase.from('orders').insert([{
  id: order.id,
  delivery_date: order.deliveryDate,
  delivery_date_label:
    order.deliveryDateLabel,
  customer_name: order.name,
```

```
    phone: order.phone,
    zone_id: order.zoneId,
    zone_name: order.zoneName,
    delivery_fee: order.deliveryFee,
    address: order.address,
    notes: order.notes,
    payment_method: order.paymentMethod,
    payment_method_label:
order.paymentMethodLabel,
    items: order.items,
    items_total: order.itemsTotal,
    grand_total: order.grandTotal,
    status: order.status,
  }]);
```

Note: the prototype also maintains a manual `order_index` array in storage (a workaround for the sandbox's key-value-only API). This is no longer needed with a real relational database — Supabase queries can list/filter orders directly. Remove the `order_index` read-modify-write logic entirely.

Loading orders for the dashboard — replace:

```
const indexResult = await
window.storage.get('order_index',
```

```
true);
```

With:

```
const { data, error } = await supabase
  .from('orders')
  .select('*')
  .order('created_at', { ascending:
false });
```

The dashboard's existing client-side logic for grouping by `delivery_date` and filtering by `status` can stay exactly as-is — it just needs `data` from the query above instead of the manually-reconstructed array from storage.

Updating order status (delivered / cancelled / restored) — replace:

```
await window.storage.set(order.id,
JSON.stringify(updated), true);
```

With:

```
await supabase
  .from('orders')
```

```
.update({ status: newStatus })  
.eq('id', order.id);
```

This single pattern covers all three status-change actions already in the UI (mark as delivered, cancel order, restore order) — they all just pass a different `newStatus` value.

Customer-side cancellation

The order confirmation screen's "Cancel order" button uses the same `window.storage.set` pattern on the order it just created — replace identically with the Supabase update pattern above.

All `window.storage` call sites are easy to find by searching the file for `window.storage`.

Dashboard PIN (recommended upgrade)

The current PIN check is a hardcoded constant near the top of the file: `const DASHBOARD_PIN = '1234';` (the client may have already changed this value — confirm with them before launch). For a production-ready upgrade, recommended approaches:

- **Simplest:** move the PIN check into a Netlify Function, store the real PIN as a Netlify environment variable, and have the frontend call the function to verify the PIN server-side rather than checking it in browser-visible code.
 - **More robust:** use Supabase Auth with a single staff login (email/password) instead of a shared PIN, gating the `/dashboard` route behind an authenticated session. Either approach removes the PIN from client-side code entirely, which is the main weakness of the current implementation.
-

6. Deployment Steps (for the developer)

1. `git init` the project, push to a new GitHub repository.
2. Sign up for Supabase, create a new project, run the SQL above to create the `orders` table and index.
3. Copy the Supabase project URL and anon key into Netlify's environment variables.

4. Connect the GitHub repo to Netlify ("New site from Git").
5. Set build command (`npm run build` if using Create React App / Vite) and publish directory (`dist` or `build`).
6. Deploy — Netlify will give a `*.netlify.app` URL immediately.
7. (Optional) Add custom domain under Netlify's Domain Settings, update DNS records as instructed.
8. Test the full flow end-to-end on the live URL:
 - Place a test order, confirm it appears on the dashboard under the correct delivery date
 - Confirm the dashboard PIN gate works correctly and the dashboard is not viewable without it
 - Mark it delivered, confirm status updates and revenue numbers behave correctly
 - Cancel a different test order, confirm it's excluded from revenue and shows correctly in the Cancelled filter
 - Test the Print Day button generates a correct, readable order sheet

9. **Strongly recommend** upgrading the dashboard PIN check from the current frontend-only constant to a proper server-side check (Supabase Auth or a Netlify Function) before considering this fully production-ready — see Section 2.
-

7. Business Context (for the developer's reference)

- Salad menu has 5 items, priced at 20,000 or 25,000 UGX (see Section 1 for exact menu and prices).
- Delivery is zone-based, calculated from Kyanja, Kampala:
 - Zone 1 (Kyanja & Surroundings): 3,000 UGX
 - Zone 2 (Near Neighbours): 4,000 UGX
 - Zone 3 (Mid Kampala): 5,000 UGX
 - Zone 4 (Far Kampala): 7,000 UGX
 - Zone 5 (Outer Kampala): 10,000 UGX
- Payment is currently either Cash on Delivery or Mobile Money to 0765119166 — there is

no automated payment verification; the rider/business confirms manually.

- Orders are placed for future delivery dates (tomorrow or up to 7 days out), not same-day – the dashboard is organized around delivery date, not order placement date, which is an intentional design choice the client requested.
- Monthly/weekly subscription ordering was discussed but is **not** part of this build – flag to the client if they ask about it, since pricing logic for that already exists in conversation history but isn't implemented in the app.
- Expected order volume currently: roughly 10–20 orders/day. Free tiers of Netlify and Supabase comfortably handle this volume; no immediate scaling concerns.

8. Estimated Effort

For a developer familiar with React + Supabase, this is realistically a **4–7 hour job** for the core migration, including testing. If unfamiliar with Supabase specifically, budget closer to a full day to account for the learning curve. Upgrading the

dashboard PIN check to proper server-side authentication (recommended, see Section 2) adds roughly **1–2 extra hours** on top.

9. Suggested Quote Range (for client negotiation reference)

Based on Uganda freelance developer rates as of mid-2026:

Hiring source	Typical total quote for this job
Local Kampala freelancer/student	100,000 – 200,000 UGX
Local agency/junior developer	180,000 – 320,000 UGX
International freelancer (Upwork etc.)	220,000 – 700,000 UGX

Recommend negotiating a **flat fee around 180,000–280,000 UGX**, paid 50% upfront and 50% on confirmed working deployment, given the slightly larger scope than the original estimate.

10. Contact / Questions

Client: Nigel — Salad Box, Kyanja, Kampala, Uganda

For questions on business logic (pricing, zones, menu, payment), refer to this spec. For anything not covered here, check with the client directly before making assumptions — particularly around upgrading the dashboard's current frontend-only PIN gate to proper server-side authentication, which should be discussed and prioritized before this handles real customer volume at scale.